ADVANCED NUMERIC PATTERN GENERATOR

Python Activity-Based Learning (ABL) Assignment — Sample Implementation

1. Problem Statement

Create a Python program that generates either a hollow numeric diamond or a symmetric numeric pattern based on user-specified dimensions. The program must validate invalid dimensions and align the output carefully.

2. Objectives

• Generate a hollow numeric diamond with a centered outline.

• Generate a symmetric numeric diamond pattern.

• Validate the menu choice and require a positive whole-number dimension.

• Handle invalid text and non-positive dimensions without crashing.

• Display neatly aligned output in the terminal.

3. Assumptions

The dimension is a positive integer. Pattern option 1 produces the hollow diamond; option 2 produces a symmetric numeric diamond. A size of 5 is used for the sample output. Very large sizes are allowed but may produce wide output.

4. Algorithm

1. Display the pattern menu.

2. Read the user's choice and validate that it is 1 or 2.

3. Read the dimension and validate that it is a positive integer.

4. For a hollow diamond, print increasing rows followed by decreasing rows, using outer and inner spaces.

5. For a symmetric pattern, print increasing number sequences and then mirror them in reverse order.

6. Display the pattern and stop.

5. Flowchart

```
START
  |
Display menu (1: Hollow, 2: Symmetric)
  |
Validate choice
  |
Read positive integer size
  |
Choose requested pattern
  |
Generate upper half and mirrored lower half
  |
Print aligned pattern
  |
 STOP
```

6. Pseudocode

```
read choice
if choice not in {1, 2}: print invalid choice; stop
read size until it is a positive integer
if choice == 1:
    print hollow diamond rows
else:
    print symmetric numeric diamond rows
```

7. Python Source Code

Save the source as **advanced_numeric_pattern_generator.py** and run with Python 3. The complete runnable source is also supplied separately.

```
"""
Advanced Numeric Pattern Generator
Generates a hollow diamond or symmetric numeric diamond.
Validates user input and aligns output carefully.
"""


def get_positive_integer(prompt):
    """Read and validate a positive integer."""
    while True:
        raw = input(prompt).strip()
        try:
            value = int(raw)
            if value <= 0:
                print("Invalid input: dimension must be a positive integer.")
                continue
            return value
        except ValueError:
            print("Invalid input: please enter a whole number.")


def generate_hollow_diamond(size):
    """Return a hollow diamond with 2*size - 1 rows."""
    rows = []
    for row in range(size):
        outer_spaces = size - row - 1
        if row == 0:
            rows.append(" " * outer_spaces + "1")
        else:
            inner_spaces = 2 * row - 1
            rows.append(" " * outer_spaces + str(row + 1) + " " * inner_spaces + str(row + 1))

    for row in range(size - 2, -1, -1):
        outer_spaces = size - row - 1
        if row == 0:
            rows.append(" " * outer_spaces + "1")
        else:
            inner_spaces = 2 * row - 1
            rows.append(" " * outer_spaces + str(row + 1) + " " * inner_spaces + str(row + 1))
    return "\n".join(rows)


def generate_symmetric_number_pattern(size):
    """Return a centered symmetric numeric pattern, increasing then decreasing."""
    lines = []
    for row in range(1, size + 1):
        left = "".join(str(n) for n in range(1, row + 1))
        right = "".join(str(n) for n in range(row - 1, 0, -1))
        lines.append(" " * (size - row) + left + right)
    for row in range(size - 1, 0, -1):
        left = "".join(str(n) for n in range(1, row + 1))
        right = "".join(str(n) for n in range(row - 1, 0, -1))
        lines.append(" " * (size - row) + left + right)
    return "\n".join(lines)


def main():
    print("=" * 48)
    print("       ADVANCED NUMERIC PATTERN GENERATOR")
    print("=" * 48)
    print("1. Hollow numeric diamond")
    print("2. Symmetric numeric diamond")
    choice = input("Choose pattern (1 or 2): ").strip()

    if choice not in ("1", "2"):
        print("Invalid choice: enter 1 or 2.")
        return

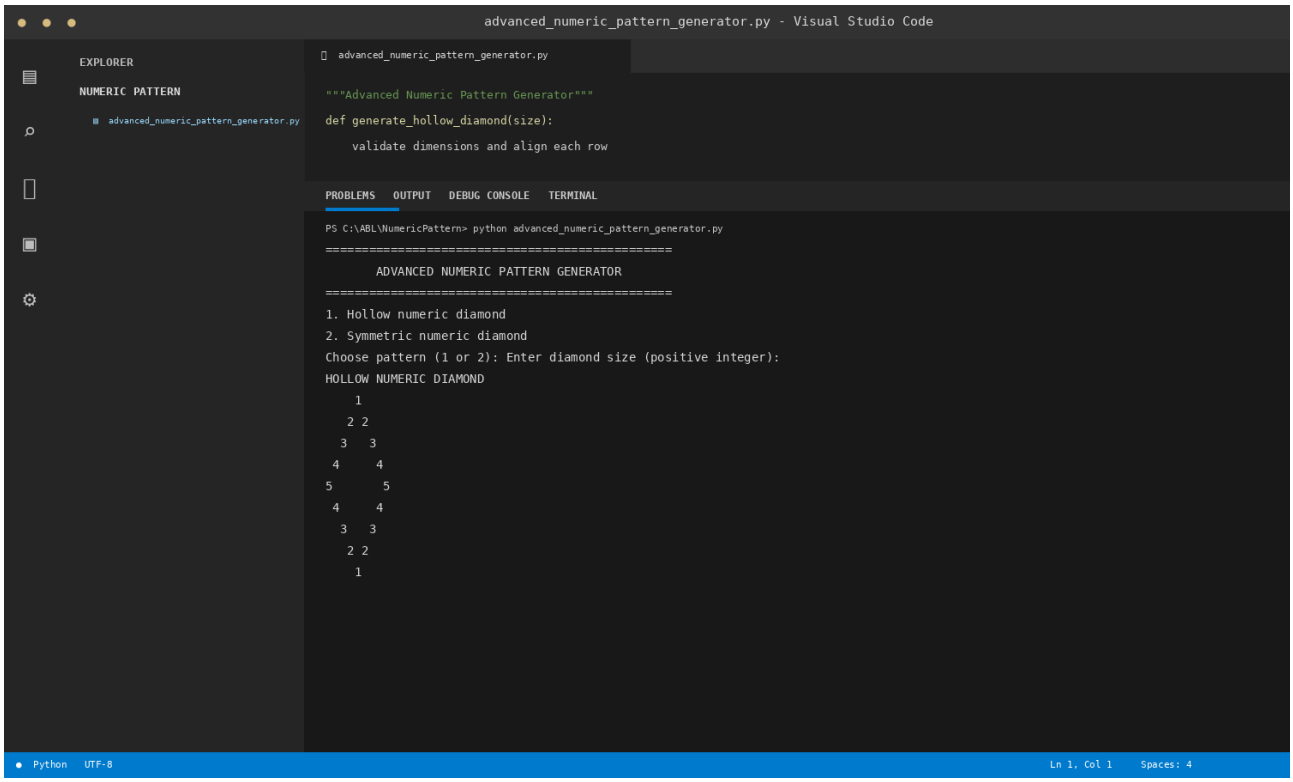
    size = get_positive_integer("Enter diamond size (positive integer): ")
    if size > 9:
        print("Warning: large dimensions may create very wide output.")

    if choice == "1":
        print("\nHOLLOW NUMERIC DIAMOND")
        print(generate_hollow_diamond(size))
    else:
        print("\nSYMMETRIC NUMERIC DIAMOND")
        print(generate_symmetric_number_pattern(size))


if __name__ == "__main__":
    main()
```

8. Sample Test and Output

Sample input: pattern choice 1 and size 5. The screenshot below is a VS Code-style mockup showing the output from executing the supplied Python file.



Terminal output (text copy)

```
==============================================
       ADVANCED NUMERIC PATTERN GENERATOR
==============================================
1. Hollow numeric diamond
2. Symmetric numeric diamond
Choose pattern (1 or 2): Enter diamond size (positive integer):
HOLLOW NUMERIC DIAMOND
    1
   2 2
  3   3
 4     4
5       5
 4     4
  3   3
   2 2
    1
```

9. Validation Test Cases

Input	Expected behavior
Choice = 1, size = 5	Print hollow numeric diamond
Choice = 2, size = 4	Print symmetric numeric diamond
Choice = 1, size = 0	Reject zero; ask for a positive integer
Choice = 1, size = abc	Reject non-numeric dimension
Choice = 3	Print invalid choice and stop

10. Conclusion

The program demonstrates nested loops, string formatting, input validation, and pattern generation. It supports two pattern types and safely rejects invalid choices and invalid dimensions.